

Pontificia Universidad Javeriana

Facultad de Ingeniería
Departamento de Ingeniería de Sistemas

Introducción a Sistemas Distribuidos

Decisiones de Diseño Gestión Inteligente de Tráfico Urbano

Integrantes:

Daniel Felipe Ramirez Vargas
Ana Sofia Arboleda
Samuel Pico
Guillermo Andrés Aponte Cárdenas

Docente: John Jairo Corredor

Fecha: 9 de abril de 2026

Índice

1. Resumen del Diseño Acordado	3
2. Decisiones Base del Proyecto	3
2.1. Supuestos Generales	3
2.2. Parámetros Configurables	3
2.3. Documentación del Archivo de Configuración	4
2.4. Organización del Código Compartido	5
2.5. Organización del Repositorio por Computador	6
3. Modelo de la Ciudad	7
3.1. Cuadrícula y Grafo Dirigido	7
3.2. Intersecciones	7
3.3. Vías	7
3.4. Nodos de Borde	8
4. Sensores y Estado del Tráfico	8
4.1. Sensores por Arista	8
4.2. Cámara	8
4.3. Espira Inductiva	8
4.4. GPS	9
4.5. Relación entre Sensores y Aristas	9
4.6. Clasificación Cualitativa de Notas	10
5. Lógica de Analítica y Semáforos	10
5.1. Score por Vía	10
5.2. Pesos y Normalización	10
5.3. Comparación entre Direcciones en Conflicto	11
5.4. Temporización Semafórica	11
5.5. Control Manual	12
6. Vehículos y Simulación	13
6.1. PC0 como Dueño del Mapa, los Vehículos y el Reloj Lógico	13
6.2. Modelo de Vehículo	13
6.3. Movimiento dentro de la Cuadrícula	13
6.4. Registro Operativo de Vehículos	14
6.5. Ambulancia	14
7. Distribución por Computadores	14
7.1. PC0	14
7.2. PC1	15
7.3. PC2	15
7.4. PC3	16

8. Persistencia y Tiempo de Simulación	16
8.1. Base de Datos en PC3	16
8.2. Réplica en PC2	16
8.3. Histórico Diario en PC0	16
8.4. Comportamiento de las Bases entre Ejecuciones	17
8.5. Reloj de Simulación	17
9. Interacción entre Componentes	18
9.1. Flujo General del Sistema	18
9.2. Snapshot Operativa	18
9.3. Creación de Ambulancias	19
9.4. Priorización Manual de Semáforos	19
10. Inicialización del Sistema	19
10.1. Configuración Inicial	19
10.2. Orden de Arranque	20
10.3. Prioridad de Implementación	20
11. Fallos y Continuidad Operativa	21
11.1. Caída de PC3	21
11.2. Continuidad con PC2	21
11.3. Backend Principal y Backend de Respaldo	21
11.4. Limitaciones Durante la Falla	22
12. Comparación de Rendimiento del Broker	22
12.1. Versión Base	22
12.2. Versión Modificada con Hilos	22

1. Resumen del Diseño Acordado

El sistema se organiza sobre cuatro computadores (PC0, PC1, PC2 y PC3) que se comunican con ZeroMQ. La ciudad se representa como una cuadrícula $N \times M$, pero internamente se maneja como un grafo dirigido donde las intersecciones son nodos y las vías son aristas con datos de tráfico. Sobre las aristas o accesos de entrada se ubican sensores lógicos de tipo cámara, espira inductiva y GPS. Sus eventos alimentan un *score* por vía de entrada antes del semáforo (véase la sección 5.1), y ese valor se usa para decidir cuál conflicto del cruce debe recibir prioridad.

En la implementación actual, **PC0 es el dueño real del mapa, de los vehículos y del reloj lógico de simulación**. PC0 genera el estado del sistema por ticks, aplica los comandos semafóricos que recibe desde PC2 y publica *las snapshots operativas* (véase la sección 9.2) para que el resto del sistema observe el mismo instante lógico. PC1 aloja los sensores y el broker ZeroMQ; PC2 ejecuta la analítica, el control semafórico y la réplica operativa; PC3 concentra la base de datos principal, el backend primario de consulta y las solicitudes activas de usuario como ambulancias o control manual.

La estrategia de resiliencia se enfoca en la caída de PC3. En ese escenario, el núcleo PC0-PC1-PC2 sigue funcionando y PC2 sostiene únicamente continuidad mínima y consulta de estado actual; no asume operaciones activas de usuario. El histórico amplio del día simulado se concentra en PC0, mientras que PC2 y PC3 se enfocan en el estado operativo presente. También se plantea un experimento de rendimiento para comparar una versión base del broker con otra mejorada con hilos.

Todo el diseño es parametrizable: el tamaño de la cuadrícula, las intersecciones activas, las frecuencias de sensores, los pesos de la analítica, los tiempos semafóricos, el reloj lógico y las reglas de tráfico se definen en archivos de configuración compartidos.

2. Decisiones Base del Proyecto

2.1. Supuestos Generales

- Se asume exactamente **un sensor lógico de cada tipo** (cámara, espira inductiva y GPS) por cada arista o acceso observado en el sistema.
- Todas las vías son de sentido único.
- El cambio de semáforo ocurre únicamente entre verde y rojo, sin fase amarilla.
- Todo el sistema se diseña de forma **parametrizada**, de modo que la escala pueda aumentarse o reducirse sin modificar la lógica central.
- La cantidad total de sensores se deriva de las aristas o accesos observados: si hay k aristas instrumentadas, existirán $3k$ sensores lógicos.

2.2. Parámetros Configurables

Los siguientes parámetros deben poder ajustarse con archivos de configuración, sin recompilar el sistema:

- Tamaño de la cuadrícula de la ciudad ($N \times M$).

- Conjunto de intersecciones activas dentro de la cuadrícula.
- Frecuencia de generación de eventos por tipo de sensor.
- Endpoints ZeroMQ (IP o nombre de host y puerto) para soportar pruebas locales o distribución real en red.
- Tiempos base de alternancia semafórica (por defecto, 15 segundos en condiciones normales).
- Reglas y umbrales que definen los estados de tráfico (normal, congestión, priorización).
- Pesos de ponderación de cada sensor para el cálculo del *score* (véase la sección 5.1).
- Parámetros del reloj de simulación (hora de inicio, hora de fin, factor de aceleración).
- Parámetros de generación de vehículos en PC0 (tasa de ingreso, velocidades, etc.).

2.3. Documentación del Archivo de Configuración

El archivo principal de configuración es `config/system_config.json`. Como el proyecto lo carga con `json.load`, no es posible usar comentarios nativos de JSON sin romper el parseo. Por esa razón se adoptó una convención explícita: incluir claves `_comentario` o `_comentarios` dentro del propio archivo para documentar el propósito de cada bloque y de cada parámetro sin afectar la ejecución.

En la implementación actual, ese archivo agrupa sus parámetros en cinco bloques principales:

- **ciudad**: define la geometría de la cuadrícula, el rango de longitud de las vías y la semilla usada para construir un mapa reproducible.
- **sensores**: define qué tipos de sensores están activos y con qué frecuencia publica PC1 sus eventos derivados de *la snapshot* (véase la sección 9.2).
- **analitica**: define el umbral general de congestión y los pesos usados para combinar cámara, espira y GPS en el *score* por vía (véase la sección 5.1).
- **simulacion**: define la duración del tick real, el tiempo simulado por tick, la frecuencia de envío de las snapshots (véase la sección 9.2), la tasa de generación de vehículos, el máximo de vehículos por tick, el rango de velocidades iniciales y la velocidad por defecto de ambulancias.
- **zmq**: define todos los endpoints de comunicación entre PC0, PC1, PC2 y PC3, incluyendo ingestas de bases de datos, broker, snapshots (véase la sección 9.2), backend principal, backend de respaldo, control manual y solicitudes de ambulancia.

Dentro del bloque `simulacion` también quedan parametrizadas de forma explícita `hora_inicio_simulada` y `hora_fin_simulada`, para que el reloj lógico del sistema no dependa solo de lo escrito en el informe sino de valores reales del archivo de configuración. En ese mismo bloque también se controla `intervalo_snapshot_ticks`,

que define cada cuántos ticks PC0 propaga la foto operativa del mapa al resto del sistema.

Esta convención tiene dos ventajas importantes para el proyecto, el archivo sigue siendo ejecutable tal como está y, al mismo tiempo, queda autoexplicado para cualquier integrante del grupo (o para el mismo ingeniero Corredor) que necesite modificar parámetros sin tener que rastrear primero el código fuente.

2.4. Organización del Código Compartido

El directorio `common` centraliza el código que debe ser reutilizado por varios procesos distribuidos del sistema. Su objetivo es reducir duplicación, evitar inconsistencias entre nodos y mantener una única fuente de verdad para contratos, modelos y utilidades compartidas.

La organización adoptada es:

- `common/mensajes`: contratos de comunicación, por ejemplo eventos, comandos, solicitudes de ambulancia y *las snapshots operativas* (véase la sección 9.2).
- `common/modelos`: entidades del dominio y del mundo simulado, por ejemplo grafo de ciudad, intersecciones, vías, vehículos y motor de simulación.
- `common/utilidades`: funciones auxiliares reutilizables, como configuración, logs, normalización de sensores, persistencia SQLite y ayudas de mensajería ZeroMQ.

Es importante aclarar que ubicar una pieza en `common` no le da autoridad operativa a cualquier computador. La autoridad sigue definida por la arquitectura: por ejemplo, PC0 es el único dueño del mapa y de los vehículos, aunque los contratos de mensajes o parte de la lógica compartida vivan en código común.

Dentro de esta organización, la carpeta `common/utilidades` cumple un papel especialmente importante porque concentra tareas transversales que deben comportarse igual en todos los computadores. Entre ellas se encuentran:

- carga de configuración compartida;
- generación de logs con formato uniforme;
- normalización y clasificación de sensores;
- persistencia SQLite compartida;
- y ayudas de mensajería ZeroMQ reutilizables.

Esta separación permite que la lógica principal del sistema quede en los modelos y en los procesos específicos de cada PC, mientras que `common/utilidades` funciona como una caja de herramientas compartida que evita reimplementar operaciones auxiliares una y otra vez.

En la implementación actual, el contenido principal de `common` puede resumirse así:

- `common/mensajes/eventos.py`: eventos de sensores emitidos por PC1, incluyendo la vía observada y el `tick_origen`.

- `common/mensajes/comandos.py`: comandos semafóricos emitidos por PC2 hacia PC0.
- `common/mensajes/estado_operativo.py`: contrato de *la snapshot operativa* (véase la sección 9.2) que representa la foto actual del sistema.
- `common/mensajes/ambulancias.py`: solicitud compartida para crear ambulancias en PC0.
- `common/modelos/trafico.py`: grafo de la ciudad, intersecciones, nodos de borde y vías.
- `common/modelos/vehiculos.py`: entidades vehiculares del sistema.
- `common/modelos/simulacion.py`: motor de simulación por ticks y reglas de movimiento.
- `common/utilidades/configuracion.py`: carga y acceso a configuración compartida.
- `common/utilidades/logs.py`: formato uniforme de logs.
- `common/utilidades/normalizacion_sensores.py`: fórmulas y clasificación de sensores.
- `common/utilidades/persistencia_sqlite.py`: persistencia compartida en SQLite.
- `common/utilidades/mensajeria_zmq.py`: ayudas ZeroMQ de mejor esfuerzo.

2.5. Organización del Repositorio por Computador

Además de `common`, el repositorio se estructura por responsabilidades de cada computador:

- `PC0/simulation`: simulación autoritativa del mapa, del tick y de los vehículos.
- `PC0/historic_db`: persistencia histórica amplia del día simulado.
- `PC1/sensors`: sensores lógicos que leen las snapshots (véase la sección 9.2) y publican eventos.
- `PC1/broker`: broker ZeroMQ base.
- `PC2/analytics`: analítica y decisión semafórica por tick.
- `PC2/traffic_ctrl`: envío de comandos hacia PC0.
- `PC2/replica_db`: réplica operativa del estado actual.
- `PC2/backend_respaldo`: backend de continuidad limitado a salud y estado presente.
- `PC3/main_db`: persistencia principal y resincronización.
- `PC3/backend`: backend primario de consulta, ambulancias y control manual.

Esta distribución busca que el repositorio exprese de manera visible la arquitectura lógica del sistema. En otras palabras, no solo separa archivos por conveniencia, sino también por responsabilidad, es decir, dónde vive la simulación, dónde se sensa, dónde se analiza, dónde se respalda el estado y dónde se expone el backend principal.

3. Modelo de la Ciudad

3.1. Cuadrícula y Grafo Dirigido

La ciudad se describe como una cuadrícula $N \times M$ (filas por letras y columnas por números). Internamente, como **decisión del grupo**, se modela como un **grafo dirigido** sobre esa cuadrícula, es decir, las intersecciones son nodos y las vías de sentido único son aristas dirigidas con atributos. Este modelo permite representar la dirección del flujo, la congestión y el estado de cada vía.

En la visualización se puede mostrar información sobre las vías como si hubiera puntos intermedios, pero en la lógica del sistema cada vía sigue siendo una arista con atributos.

3.2. Intersecciones

Cada intersección es un nodo del grafo con los siguientes atributos:

- **Identificador único:** por ejemplo INT-C5 (fila C, columna 5).
- **Coordenada** (*fila, columna*).
- **Sensores relacionados:** las aristas o accesos conectados a la intersección cuentan con sensores lógicos de cámara, espira inductiva y GPS.
- **Dos semáforos lógicos:** eje horizontal y eje vertical.
- **Fase activa:** HORIZONTAL o VERTICAL; nunca hay dos fases en verde simultáneamente.

3.3. Vías

Cada vía se modela como una arista dirigida del grafo con los siguientes atributos. Como todas las vías son unidireccionales, cada arista representa **un solo sentido de circulación**:

- Intersección origen.
- Intersección destino (o nodo de borde destino).
- Longitud o costo de la vía.
- Dirección cardinal del movimiento (NORTE, SUR, ESTE u OESTE).
- Vehículos en circulación dentro de la arista.
- Vehículos en espera (detenidos por semáforo en rojo al final de la arista).
- Velocidad promedio observada.
- *Score* de congestión (véase la sección 5.1), como valor normalizado en $[0, 1]$.
- Estado de congestión derivado del *score* (véase la sección 5.1): tráfico normal, congestión o priorización.

3.4. Nodos de Borde

Los bordes de la cuadrícula se modelan mediante **nodos de borde**, que permiten representar tanto la entrada como la salida de vehículos de la ciudad (o del fragmento simulado). Sus atributos son:

- Identificador único.
- Intersección del borde a la que está conectado.
- Lado del borde: NORTE, SUR, ESTE u OESTE.

Estos nodos habilitan el ingreso de vehículos generados por PC0 y el egreso cuando un vehículo alcanza el borde de la ciudad. La convención adoptada usa números de columna para norte/sur (por ejemplo BORDE-S-1) y letras de fila para oeste/este (por ejemplo BORDE-O-A).

4. Sensores y Estado del Tráfico

4.1. Sensores por Arista

Cada arista o acceso instrumentado cuenta con sensores lógicos de cada tipo: cámara, espira inductiva y GPS. En esta solución, los sensores no se entienden como dispositivos ubicados físicamente en el punto de la intersección, sino como **observadores de aristas o accesos**. En la práctica, funcionan como **getters** del estado de la arista: consultan de forma periódica los atributos del tramo observado y procesan esos datos para producir las estadísticas y eventos que necesita la analítica. Como la vía tiene un único sentido, el sensor solo mide el flujo que viene en ese sentido hacia el semáforo.

4.2. Cámara

La cámara mide la acumulación o cola de vehículos y aporta una señal de **ocupación inmediata**. Sus datos salen de leer la cantidad de vehículos en espera al final de la arista observada (`EVENTO_LONGITUD_COLA`, L_q), es decir, los vehículos que ya completaron la longitud de la vía y se encuentran detenidos antes de la intersección.

La nota de prioridad aportada por la cámara se calcula como:

$$n_c = \min \left(\frac{\text{vehículos en espera}}{10}, 1 \right) \quad (1)$$

Esto implica que 1 vehículo en espera aporta 0.1, 5 vehículos aportan 0.5 y 10 o más aportan 1.0.

4.3. Espira Inductiva

La espira inductiva mide la cantidad de vehículos que están en tránsito dentro de la arista antes de llegar a la intersección y aporta una señal de **presión de tráfico** (`EVENTO_CONTEO_VEHICULAR`, C_v). En esta versión del diseño, la espira **no mide la cola**; solo mide vehículos que todavía recorren la arista.

La nota de prioridad aportada por la espira se calcula como:

$$n_e = \min\left(\frac{\text{vehículos en tránsito}}{10}, 1\right) \quad (2)$$

Esto implica que 1 vehículo en tránsito aporta 0.1, 5 vehículos aportan 0.5 y 10 o más aportan 1.0.

4.4. GPS

El sensor GPS mide la velocidad promedio de los vehículos **en tránsito** en una arista y aporta una señal complementaria de **fluidez o lentitud** (`EVENTO_DENSIDAD_DE_TRAFICO`, D_t). Como decisión del grupo, el GPS **no se usa como criterio único** para cambiar semáforos, sino como un componente ponderado dentro del *score* total (véase la sección 5.1).

Si no hay vehículos en tránsito, el promedio de velocidad es 0 y la nota del GPS también es 0. Si sí hay vehículos circulando, el promedio queda entre 10 y 50 porque cada vehículo mantiene una velocidad propia dentro de ese rango mientras se mueve por la vía.

La nota del GPS se calcula por tramos:

- Si $x = 0$, entonces $n_g = 0$.
- Si $10 \leq x \leq 50$, entonces:

$$n_g = 1 - 0,9 \cdot \frac{x - 10}{40} \quad (3)$$

Con esta normalización, 10 se mapea a 1.0 y 50 se mapea a 0.1. En otras palabras, a menor velocidad promedio de tránsito, mayor prioridad aporta el GPS.

4.5. Relación entre Sensores y Aristas

Los sensores simulados en PC1 están definidos sobre aristas o accesos y consultan el estado de los vehículos mantenido por PC0 para generar eventos realistas. En otras palabras, cada sensor actúa como un getter especializado del estado de una vía: toma atributos del tramo observado, los procesa según su tipo y publica la estadística resultante. Así, la simulación en PC0 alimenta de forma directa el flujo de datos del resto del sistema.

En la implementación actual, los eventos se emiten **por arista instrumentada** y cada evento identifica explícitamente la vía observada, la intersección destino y el `tick_origen` de la simulación. Esto permite que PC2 agrupe lecturas coherentes del mismo instante lógico antes de calcular una decisión semafórica.

4.6. Clasificación Cualitativa de Notas

Las notas normalizadas de los sensores se interpretan también cualitativamente con la misma escala. Esta clasificación se aplica de manera uniforme a las notas parciales de cámara, espira inductiva y GPS, de modo que los valores numéricos no solo sirvan para calcular el *score* (véase la sección 5.1), sino también para comunicar de forma más clara el nivel de tráfico observado en cada vía:

- **Bajo:** $0,0 \leq x < 0,4$
- **Moderado:** $0,4 \leq x \leq 0,7$
- **Intenso:** $0,7 < x \leq 1,0$

Esta escala facilita la interpretación humana del sistema y también sirve como base natural para una futura visualización del mapa, donde una categoría cualitativa resulta más fácil de mostrar y discutir que una nota continua aislada.

5. Lógica de Analítica y Semáforos

5.1. Score por Vía

La lógica de control semafórico se basa en una **calificación de estado** (*score*) para cada vía de entrada a una intersección. Ese *score* solo describe la carga del tráfico **antes del semáforo** y en el **único sentido** permitido de la vía. Cada sensor aporta al *score* con una ponderación configurable. Entre mayor sea el *score*, mayor será la prioridad para recibir verde. El valor final queda normalizado en $[0, 1]$. Los tres sensores aportan de forma conjunta, sin depender de un solo indicador.

5.2. Pesos y Normalización

Cada sensor produce primero una **nota normalizada** entre 0 y 1, de acuerdo con el estado observado en la arista. Después, esas tres notas se combinan con pesos configurables para obtener un *score* final, también entre 0 y 1.

Si se denotan las notas de cámara, espira y GPS como n_c , n_e y n_g , entonces:

$$n_c, n_e, n_g \in [0, 1] \quad (4)$$

$$w_c, w_e, w_g \in [0, 1] \quad (5)$$

$$w_c + w_e + w_g = 1 \quad (6)$$

Para esta versión del diseño, se fijan los siguientes pesos:

- $w_c = 0,50$: peso de la **cámara**. Representa la importancia de la cola o acumulación de vehículos en la vía.
- $w_e = 0,35$: peso de la **espira inductiva**. Representa la importancia del flujo de vehículos que pasan por la vía.
- $w_g = 0,15$: peso del **GPS**. Representa la importancia de la velocidad promedio como señal complementaria de fluidez o lentitud.

Con estos valores, el *score* de la vía se calcula como:

$$\text{score}_{\text{via}} = w_c n_c + w_e n_e + w_g n_g \quad (7)$$

Como los pesos suman 1 y cada nota está en $[0, 1]$, el resultado final también queda en el rango $[0, 1]$. En esta distribución, la cámara pesa más porque la cola de vehículos es la señal más importante para decidir prioridad; la espira tiene un peso intermedio porque muestra presión de flujo; y el GPS pesa menos porque solo complementa la lectura de congestión. Estos valores pueden ajustarse más adelante si el grupo decide recalibrar el sistema.

5.3. Comparación entre Direcciones en Conflicto

Cada intersección se controla comparando las vías de entrada que están en conflicto antes del semáforo. El *score* no se calcula por eje como valor base, sino por cada vía de entrada. Por ejemplo, una intersección puede tener:

- una vía que llega desde el norte con su propio *score*,
- una vía que llega desde el sur con su propio *score*,
- una vía que llega desde el este con su propio *score*,
- una vía que llega desde el oeste con su propio *score*.

Con esos valores, el servicio de analítica en PC2 compara las direcciones que compiten por el paso en el cruce. La dirección con mayor *score* recibe prioridad. Si los puntajes son parecidos, se mantiene la alternancia normal o se aplica una regla de desempate configurable.

En la práctica, el servicio de analítica en PC2 revisa de manera continua los conflictos de todas las intersecciones del grafo. Para cada nodo identifica las vías de entrada que compiten por el paso, consulta sus *scores* actuales y decide cuál debe recibir prioridad semafórica en ese momento. Como el control final se expresa por ejes lógicos (HORIZONTAL y VERTICAL), la implementación obtiene el score por eje como el promedio de los scores de las vías de entrada asociadas a ese eje.

5.4. Temporización Semafórica

La duración del verde se ajusta según la diferencia entre los *scores* de las direcciones que están en conflicto. Se toma como base un ciclo total de **30 segundos**, porque en condiciones normales cada dirección recibe 15 segundos antes del cambio, tal como indica el enunciado. En esta simulación, esos 15 segundos son segundos reales de ejecución, pero equivalen a 15 minutos dentro del tiempo simulado de la ciudad. Se define:

$$\text{gap} = |\text{score}_1 - \text{score}_2| \in [0, 1]$$

- Si $\text{gap} = 0$: no existe diferencia de prioridad entre las dos direcciones comparadas; cada una recibe **15 segundos**, completando un ciclo total de 30 segundos.

- Si $0 < \text{gap} < 1$: la dirección con mayor *score* recibe una porción mayor del ciclo total de 30 segundos. Entre mayor sea el *gap*, mayor será el tiempo en verde para esa dirección y menor para la dirección opuesta.
- Si $\text{gap} = 1$: significa que una dirección tiene *score* mínimo y la otra *score* máximo. En la práctica, esto solo ocurre si una de las dos direcciones tiene *score* 0, es decir, no tiene carga vehicular antes del semáforo. En ese caso, la dirección ganadora recibe los **30 segundos completos** del ciclo.

De forma general, si $\text{score}_{\text{max}}$ es el mayor de los dos *scores* comparados, el tiempo de verde de la dirección prioritaria puede expresarse como:

$$T_{\text{verde}} = 15 + 15 \cdot \text{gap} \quad (8)$$

De este modo:

- con $\text{gap} = 0$, $T_{\text{verde}} = 15$ segundos;
- con $\text{gap} = 1$, $T_{\text{verde}} = 30$ segundos.

El tiempo restante del ciclo queda asignado a la dirección opuesta:

$$T_{\text{opuesto}} = 30 - T_{\text{verde}} \quad (9)$$

Además de esta temporización, la implementación actual aplica dos reglas operativas importantes:

- **Deduplicación de comandos:** si para una intersección la fase y los tiempos calculados no cambian respecto a la última orden emitida, el sistema no reenvía exactamente el mismo comando.
- **Una decisión por intersección y por tick:** aunque los eventos lleguen secuencialmente, PC2 agrupa las mediciones del mismo `tick_origen` y emite como máximo una decisión por intersección en ese instante lógico.

5.5. Control Manual

Desde PC3 el usuario puede forzar manualmente el estado de un semáforo:

- Selecciona una intersección y un eje a priorizar.
- Indica por cuánto tiempo de simulación mantener el forzado.
- Mientras dure, la lógica automática basada en *score* queda suspendida en esa intersección.
- Al finalizar, la intersección regresa al control automático.

El forzado se expresa en tiempo de simulación y no en tiempo real.

6. Vehículos y Simulación

6.1. PC0 como Dueño del Mapa, los Vehículos y el Reloj Lógico

Como **extensión del grupo de computadores**, se incorpora PC0 para la simulación de vehículos que cambian el estado del tráfico. En la implementación actual, PC0 no es solo un generador de vehículos: es el **dueño real del mapa, del estado de las vías, de los vehículos y del reloj lógico de simulación**. Desde PC0 se generan vehículos que entran a la ciudad por nodos de borde, se aplican los comandos semafóricos que llegan desde PC2 y se calcula el estado que será propagado al resto del sistema.

Además, PC0 publica *las snapshots operativas* (véase la sección 9.2) con el estado actual hacia PC1, para que los sensores lean esa foto del sistema sin duplicar la simulación. PC0 también almacena el histórico completo de un día simulado para análisis posterior y comunica el estado presente a la base principal de PC3 y a la réplica operativa de PC2.

6.2. Modelo de Vehículo

Cada vehículo es una entidad simulada con identificador único. Un vehículo solo puede estar en una arista a la vez. Sus atributos mínimos son:

- Identificador del vehículo.
- Tipo de vehículo (**NORMAL** o **AMBULANCIA**).
- Arista actual en la que se encuentra.
- Posición relativa o progreso dentro de la arista.
- Velocidad simulada.
- Dirección actual de movimiento.
- Estado actual (**CIRCULANDO**, **EN_ESPERA** o saliendo del sistema).
- *Timestamp* de última actualización (en tiempo de simulación).

6.3. Movimiento dentro de la Cuadrícula

Los vehículos entran al sistema por un nodo de borde y recorren aristas dirigidas entre intersecciones. Al llegar a una intersección, el vehículo decide aleatoriamente entre seguir derecho o tomar la alternativa permitida. No puede moverse en contra del sentido de una vía. Sale del sistema cuando llega a un nodo de borde configurado como egreso.

La presencia y el movimiento de los vehículos cambian el estado de las vías: cada vehículo aporta al conteo de vehículos en circulación dentro de su arista. Si el semáforo de su eje está en rojo y el vehículo llega al final de la arista, aporta al conteo de vehículos en espera. Estos valores alimentan las mediciones de los sensores simulados.

La velocidad de tránsito se asigna al vehículo cuando se instancia y permanece como atributo propio del vehículo. Cuando el carro logra pasar por una intersección con semáforo en verde y entra a una nueva vía, continúa recorriendo esa nueva arista

con la misma velocidad que ya tenía asignada. Cuando un vehículo sale de la ciudad, se elimina del estado activo del motor de simulación.

6.4. Registro Operativo de Vehículos

Para facilitar pruebas sin interfaz gráfica, PC0 registra en logs la creación y la salida de vehículos de la simulación. Este registro operativo no reemplaza la persistencia en base de datos, pero sí ofrece una traza inmediata y legible de lo que está ocurriendo en cada tick del motor de simulación.

En los logs de creación se reportan, entre otros, el identificador, el tipo, la vía inicial, el origen, el destino de la vía, la dirección, la velocidad, el estado, el tick y la hora simulada. Esto permite verificar que la generación de tráfico respeta la configuración del sistema y que los vehículos nacen en los nodos de borde correctos.

En los logs de salida se reporta también el motivo de retiro del vehículo del estado activo, por ejemplo salida efectiva de la ciudad o ausencia de una ruta de continuación válida. De esta forma, cuando un vehículo deja de existir dentro de la simulación, esa eliminación no queda implícita sino registrada de forma explícita.

Como complemento, el resumen de cada tick incluye la hora simulada correspondiente al estado actual del motor de simulación lo cual permite relacionar creación, movimiento y salida de vehículos con el reloj lógico propagado por PC0.

6.5. Ambulancia

Desde PC3 el usuario puede solicitar manualmente una ambulancia en un nodo de borde. La ambulancia cuenta como un vehículo más, pero con una representación visual distinta y una velocidad constante configurable. A medida que avanza, el usuario puede intervenir manualmente los semáforos desde PC3 para abrirle paso. La ambulancia sale del sistema al llegar a un nodo de borde.

Operativamente, la ambulancia **la solicita PC3 y la crea PC0**. El backend principal de PC3 construye la solicitud y la envía por ZeroMQ a PC0; PC0 recibe esa solicitud y la inyecta en el motor de simulación como un vehículo de tipo **AMBULANCIA**. En una futura visualización basta con conservar su tipo, su vía actual y su posición por tick para distinguirla del tráfico normal.

7. Distribución por Computadores

7.1. PC0

Extensión propuesta al grupo de computadores. PC0 es el dueño del estado autoritativo de la simulación y mantiene el estado base del tráfico. Sus responsabilidades son:

- Generar vehículos que ingresan a la ciudad por nodos de borde.
- Mantener y actualizar la posición de cada vehículo en el grafo.
- Mantener el estado autoritativo del mapa, las vías y los semáforos dentro de la simulación.

- Instanciar ambulancias cuando PC3 lo solicite por ZeroMQ.
- Recibir desde PC2 los comandos semafóricos automáticos y aplicarlos sobre la simulación.
- Publicar las snapshots operativas del estado actual (véase la sección 9.2) hacia PC1.
- Comunicar el estado de los vehículos en el grafo-mapa a la base de datos principal de PC3, a la réplica de PC2 y a su propia base histórica en PC0.
- Almacenar el historial completo de un día de simulación para estadísticas finales.

PC0 **no** forma parte del mecanismo principal de respaldo cuando ocurre una falla; su almacenamiento histórico es para análisis posterior, no para mantener la operación del sistema.

7.2. PC1

PC1 aloja los sensores y el broker. Sus responsabilidades son:

- Ejecutar los sensores simulados (cámara, espira inductiva y GPS) como procesos lógicos asociados a aristas que generan eventos periódicos.
- Recibir las snapshots operativas producidas por PC0 (véase la sección 9.2) y usarlas como fuente de verdad para calcular sus mediciones.
- Publicar eventos mediante PUB/SUB de ZeroMQ, con tópicos diferenciados por tipo de sensor.
- Operar el broker ZeroMQ que recibe los eventos y los reenvía a PC2.

7.3. PC2

PC2 aloja la analítica y el control de semáforos. Sus responsabilidades son:

- Suscribirse a los eventos de sensores vía el broker de PC1.
- Calcular el *score* por vía y determinar la fase semafórica de cada intersección.
- Emitir como máximo un comando por intersección y por tick de simulación, agrupando mediciones del mismo *tick_origen*.
- Ejecutar órdenes de control sobre semáforos e imprimir por pantalla las acciones realizadas.
- Recibir y ejecutar indicaciones de control manual provenientes de PC3.
- Mantener la **réplica de la base de datos**, actualizada de forma asíncrona, para que el sistema pueda seguir operando si PC3 falla.
- Exponer un backend de respaldo limitado a salud y consultas de estado actual, sin crear ambulancias ni aceptar control manual durante el failover.

7.4. PC3

PC3 aloja el monitoreo, la consulta y la persistencia principal. Sus responsabilidades son:

- Alojar la **base de datos principal**.
- Prover monitoreo y consulta del estado actual mediante REQ/REP.
- Enviar indicaciones directas al servicio de analítica para forzar cambios semafóricos.
- Exponer el backend primario que el cliente consulta normalmente antes de considerar el respaldo de PC2.
- Visualizar la ciudad como grafo sobre cuadrícula, con vías coloreadas según congestión e indicadores de fase activa (extensión del grupo de computadores).
- Permitir al usuario solicitar ambulancias en nodos de borde.

8. Persistencia y Tiempo de Simulación

8.1. Base de Datos en PC3

La base de datos principal reside en PC3. Su responsabilidad principal es almacenar el **estado operativo actual** de la ciudad en tiempo real: estado de intersecciones, estado de vías, estado actualizado de vehículos y estado vigente de semáforos.

PC3 **no** se concibe como el repositorio principal de histórico amplio de eventos de sensores ni de comandos semafóricos. Ese histórico se concentra en PC0. La base principal de PC3 se enfoca en el estado presente del sistema y en servir como soporte del backend primario.

8.2. Réplica en PC2

La réplica se encuentra en PC2 y se actualiza de forma asíncrona (PUSH/PULL u otro patrón similar). Su propósito es mantener el **estado operativo actual** de la ciudad y el estado de los vehículos reportado desde PC0, de modo que el sistema pueda seguir funcionando si PC3 falla. PC2 no se plantea como almacén de resultados históricos de largo plazo, sino como respaldo del estado presente.

Si PC3 cae, el núcleo operativo **PC0-PC1-PC2** sigue funcionando. Cuando PC3 vuelve a estar disponible, su base de datos se actualiza nuevamente con el estado que tenía PC2 y, una vez termina esa sincronización, el sistema vuelve a operar usando PC3 como base principal. Los envíos a PC3 se manejan en mejor esfuerzo para no bloquear el ciclo principal si la base principal no está disponible.

8.3. Histórico Diario en PC0

PC0 almacena el historial completo de un día de simulación para análisis posterior y estadísticas finales (extensión del grupo de computadores). En esta base también quedan registrados los eventos de sensores, los comandos semafóricos y el estado histórico de los vehículos a lo largo del día simulado. PC0 **no** se utiliza como nodo de resiliencia operativa; su rol es exclusivamente analítico.

8.4. Comportamiento de las Bases entre Ejecuciones

Si el usuario detiene una ejecución y luego vuelve a arrancar el sistema sin borrar las bases SQLite, no todos los computadores se comportan igual:

- La simulación **no se reanuda** desde base de datos. Cada vez que PC0 arranca, construye un mapa nuevo y un motor nuevo en memoria.
- La base histórica de PC0 **sí acumula** datos entre corridas si no se limpia previamente.
- Las bases de PC2 y PC3 conservan el último estado actual persistido, pero ese estado no gobierna la nueva simulación ya que se reemplaza con las snapshots nuevas (véase la sección 9.2) que emita PC0.
- Mientras no llegue la primera snapshot nueva (véase la sección 9.2), una consulta temprana a PC2 o PC3 todavía podría mostrar estado viejo persistido de la corrida anterior.
- Si PC3 arranca antes de que la nueva simulación emita snapshots (véase la sección 9.2), puede resincronizarse temporalmente con la última snapshot vieja almacenada en PC2; eso se corrige en cuanto PC0 vuelve a emitir estado nuevo.

En consecuencia, el comportamiento entre ejecuciones combina dos efectos distintos: por un lado, el motor activo siempre comienza una simulación nueva en memoria; por otro, las bases pueden conservar rastros de la corrida anterior hasta que llegan nuevas actualizaciones o hasta que el usuario las limpia manualmente.

Esto se vuelve especialmente visible cuando PC3 arranca antes de que exista una nueva corrida activa de PC0. En ese caso, PC3 no se queda esperando indefinidamente a que llegue una snapshot nueva (véase la sección 9.2), sino que intenta recuperar de inmediato el estado actual desde PC2. Como PC2 responde con la última foto operativa que tenga guardada, esa respuesta puede corresponder todavía a la ejecución anterior si PC0 aún no ha empezado a emitir estados nuevos y actualizados.

Por esta razón, si se quiere una prueba completamente limpia y sin mezclar corridas anteriores, deben borrarse previamente las SQLite de PC0, PC2 y PC3. Para facilitar esa operación, el proyecto incluye un script de limpieza dedicado dentro del directorio `scripts`.

8.5. Reloj de Simulación

El sistema comparte un reloj global de simulación que representa un día de **12:00 a 18:00**. Por defecto, para efectos de la simulación, **1 segundo de tiempo real representa 1 minuto de tiempo simulado**. Esto implica que un cambio semafórico normal de 15 segundos en tiempo real equivale a 15 minutos dentro de la ciudad simulada.

En la implementación actual, el reloj lógico se materializa en **PC0**. Cada tick del motor incrementa el tiempo simulado y esa referencia temporal se propaga al resto del sistema por medio de las snapshots operativas (véase la sección 9.2) y del `tick_origin` en los eventos. Todos los eventos (sensores, vehículos, semáforos y persistencia) usan el tiempo de simulación, no el tiempo real.

El rango horario queda también parametrizado en `config/system_config.json` mediante `hora_inicio_simulada` y `hora_fin_simulada`. Adicionalmente, la cadencia

de propagación del estado del mapa se controla con `intervalo_snapshot_ticks`. Por ahora, la hora final se usa como referencia visible del día simulado y para enriquecer logs, pero no detiene automáticamente la simulación.

9. Interacción entre Componentes

9.1. Flujo General del Sistema

El flujo operativo del sistema sigue esta cadena:

1. **PC0** genera vehículos y mantiene el estado base del tráfico.
2. **PC0** comunica el estado actualizado a **PC3** (BD principal), a **PC2** (réplica operativa) y a **PC0** (base histórica).
3. **PC1** recibe las snapshots de **PC0** (véase la sección 9.2), ejecuta sensores y publica eventos a través del broker ZeroMQ.
4. **PC2** se suscribe a los eventos, calcula analítica por vía, agrupa scores por eje y determina fases semafóricas.
5. **PC3** permite monitorear, consultar y emitir indicaciones de control manual.

9.2. Snapshot Operativa

Una *snapshot operativa* puede entenderse como una foto del estado actual del sistema en un tick determinado. No representa una película completa del tráfico, sino un corte puntual del mundo simulado en un instante lógico. Esa snapshot contiene al menos el estado presente de intersecciones, vías y vehículos, junto con el tick actual y la marca temporal de simulación.

En términos prácticos, la snapshot sirve para que:

- **PC0 le diga a PC1**: “así está el mundo ahorita”.
- **PC1** use esa foto para calcular sensores.
- **PC2** y **PC3** guarden estado actual.
- **PC3** pueda resincronizarse desde **PC2** si volvió a levantarse.

Desde el punto de vista de arquitectura, la snapshot operativa es el contrato que evita que **PC1**, **PC2** y **PC3** tengan que reconstruir por su cuenta el estado del mapa. En lugar de duplicar la simulación, estos componentes consumen la foto autoritativa producida por **PC0** y trabajan a partir de ella.

La cadencia de envío de la snapshot se controla por configuración. En la implementación actual, **PC0** emite una primera snapshot en el primer tick de simulación y, a partir de ahí, vuelve a emitirla cada N ticks según el valor configurado en la siguiente clave: `simulacion.intervalo_snapshot_ticks`

Esto permite balancear dos necesidades opuestas, es decir, por un lado, una observación más fina del sistema; por otro, menor volumen de mensajes y menor presión de persistencia.

Los endpoints ZeroMQ asociados a la propagación de estado también se definen por configuración. Durante pruebas locales pueden usarse direcciones de `localhost`, pero en despliegues sobre varios computadores esos endpoints deben reemplazarse por las IP o nombres reales de las máquinas participantes, sin necesidad de modificar la lógica del sistema.

9.3. Creación de Ambulancias

Cuando el usuario crea una ambulancia desde la interfaz de PC3:

1. PC3 envía la solicitud por ZeroMQ a PC0, indicando el nodo de borde donde debe aparecer la ambulancia.
2. PC0 instancia la ambulancia como una entidad vehicular especial dentro de la simulación.
3. PC3 la visualiza con una representación diferenciada (por ejemplo, icono de sirena) respecto al tráfico normal.

La creación de la ambulancia recae en PC0 y la visualización/control en PC3. Es decir: **PC3 la pide por ZeroMQ y PC0 la crea realmente**. En la implementación operativa esto se modela con un canal dedicado donde el backend principal de PC3 emite una solicitud de ambulancia y PC0 la valida e inyecta en el motor de simulación.

9.4. Priorización Manual de Semáforos

Cuando el usuario fuerza un semáforo desde PC3:

1. PC3 envía la orden al servicio de analítica/control en PC2.
2. La orden indica qué intersección y qué eje priorizar, y por cuánto tiempo de simulación mantener el forzado.
3. PC2 ejecuta el forzado, suspendiendo temporalmente la lógica automática en esa intersección.
4. Al finalizar el período indicado, la intersección regresa automáticamente al control por *score*.

En la implementación actual, este forzado se realiza por ZeroMQ: el backend principal de PC3 emite la solicitud de control manual, PC2 la valida y genera un comando semafórico manual que envía inmediatamente a PC0, donde se refleja en el siguiente ciclo operativo de la simulación. Si PC3 no está disponible, el backend de respaldo de PC2 no acepta nuevos controles manuales.

10. Inicialización del Sistema

10.1. Configuración Inicial

Se define un archivo, o un conjunto de archivos, de configuración compartidos que todos los componentes leen al arrancar. Esta configuración incluye al menos:

- Tamaño de la cuadrícula ($N \times M$).
- Intersecciones activas y nodos de borde.
- Número y tipo de sensores por arista o acceso instrumentado.
- Parámetros de semáforos (tiempo base y duración del ciclo).
- Pesos de ponderación de sensores y umbrales de analítica.
- Parámetros del reloj de simulación (hora inicio, hora fin, factor de velocidad).
- Parámetros de generación de vehículos (tasa de ingreso, velocidades).

Esta configuración centralizada permite que el arranque sea reproducible y que los parámetros se ajusten sin cambiar código.

10.2. Orden de Arranque

El orden de arranque acordado es:

1. **PC3**: levanta la base de datos principal y el backend primario.
2. **PC2**: levanta el servicio de analítica, el control semafórico y la réplica operativa de la base de datos.
3. **PC1**: levanta los sensores simulados y el broker ZeroMQ.
4. **PC0**: inicia la simulación autoritativa, la generación de vehículos y el almacenamiento histórico diario.

Este orden garantiza que los componentes consumidores y de persistencia estén disponibles antes de empezar a emitir eventos y a mover vehículos.

10.3. Prioridad de Implementación

Para la implementación incremental del proyecto se prioriza primero el núcleo operativo formado por **PC0**, **PC1** y **PC2**. En esta etapa se busca completar:

- simulación vehicular y estado base del mapa en PC0,
- sensado y broker en PC1,
- analítica, control semafórico y réplica operativa en PC2.

Esta decisión permitió estabilizar primero los elementos más delicados del sistema, es decir, el movimiento de vehículos por ticks, la medición correcta sobre aristas, el cálculo del score por vía, la emisión controlada de comandos semafóricos, la persistencia del estado actual y la propagación de la snapshot operativa entre computadores. De esta manera, las capas posteriores no se construyeron sobre un comportamiento todavía incierto o inconsistente.

Una vez estabilizado el flujo central PC0-PC1-PC2, se añade el backend principal de PC3 y el backend de respaldo reducido en PC2, enfocado solamente en continuidad y consulta del estado presente, dejando para una etapa posterior la visualización (front-end) del mapa.

11. Fallos y Continuidad Operativa

11.1. Caída de PC3

La falla principal considerada es la caída de PC3. Si PC3 falla, el backend principal deja de responder, pero el sistema no pierde el control operativo porque PC0, PC1 y PC2 continúan ejecutando la simulación, el sensado, la analítica y la réplica de estado. Desde el punto de vista del usuario, se pierden temporalmente:

- Visualización en tiempo real del mapa de la ciudad.
- Monitoreo interactivo y consultas desde la interfaz de usuario.
- Creación de nuevas ambulancias.
- Emisión de órdenes manuales de priorización semafórica.

11.2. Continuidad con PC2

Ante la caída de PC3, el sistema sigue operando con la réplica en PC2. Los componentes internos nunca dejan de trabajar y el respaldo puede exponer como máximo salud y consulta de estado actual para observación mínima.

Si PC3 se vuelve a levantar, el sistema no se queda indefinidamente en PC2. En ese caso, se sincroniza de nuevo la base de datos de PC3 con el estado actual que tiene PC2 y, al terminar esa actualización, la operación vuelve automáticamente a PC3 como base principal.

11.3. Backend Principal y Backend de Respaldo

El sistema distingue dos roles de backend:

- **Backend principal en PC3:** es el camino normal para consultas de estado, creación de ambulancias y control manual.
- **Backend de respaldo en PC2:** comparte el mismo protocolo de solicitudes, pero queda restringido a continuidad mínima, salud y estado actual.

La operación mínima de continuidad incluye también una consulta de **salud**. Esta consulta funciona como un *ping* del backend y permite saber qué servidor atendió la solicitud. Si responde PC3, el cliente sigue en el camino normal; si PC3 no responde y la petición cae a PC2, la respuesta identifica explícitamente al respaldo como backend activo.

El cliente de failover intenta primero hablar con PC3; si no recibe respuesta, reintenta automáticamente la misma solicitud contra PC2. Esta decisión hace que el respaldo mantenga continuidad de protocolo, es decir, que pueda seguir respondiendo dentro del mismo esquema de solicitudes sin obligar al cliente a cambiar de interfaz o de contrato.

Sin embargo, compartir el mismo protocolo no significa compartir la misma autoridad operativa. El respaldo no ejecuta operaciones activas. Si recibe solicitudes como las siguientes:

```
crear_ambulancia  
control_manual
```

responde explícitamente que la operación no está disponible en modo respaldo. Ese rechazo es deliberado y forma parte del diseño, para evitar que la continuidad mínima de PC2 se convierta en una segunda capa completa de control de usuario. Aunque el cliente de failover podría reenviar esas solicitudes al respaldo, solo PC3 está autorizado para iniciarlas como operaciones activas de usuario.

11.4. Limitaciones Durante la Falla

Durante la falla de PC3, las siguientes funcionalidades quedan indisponibles:

- No se pueden crear nuevas ambulancias desde la interfaz.
- No se pueden emitir nuevas órdenes manuales de priorización semafórica.
- Se pierde la visualización y el monitoreo interactivo.

PC0 **no** participa en el cambio a un respaldo cuando ocurre una falla. El escenario de resiliencia es: PC3 cae y el sistema sigue operando con la réplica de PC2.

Durante el proceso de resincronización de PC3 pueden seguir ocurriendo cambios en el sistema mientras se copia el estado desde PC2. Por esta razón, se acepta que al volver a operar con PC3 pueda aparecer una pequeña inconsistencia temporal o una leve sensación de “retroceso en el tiempo” en la simulación. Esta limitación se considera aceptable dentro del alcance del proyecto.

12. Comparación de Rendimiento del Broker

12.1. Versión Base

Primera versión del broker en PC1 con una lógica simple y sin concurrencia interna. Recibe eventos y los reenvía a PC2 de manera secuencial. Esta es la línea base del experimento de rendimiento.

12.2. Versión Modificada con Hilos

Segunda versión del broker que introduce **hilos** para separar la recepción, el encolado y el reenvío de eventos en paralelo.

Métricas de comparación entre ambas versiones:

- Cantidad de eventos almacenados en la BD en una ventana de 2 minutos.
- Tiempo desde que el usuario solicita una acción hasta que el semáforo cambia.

Los escenarios de prueba varían el número de sensores y el tiempo entre generación de mediciones.